2026 IEEE International Conference on Artificial Intelligence Testing (AITest) | 979-8-3195-0562-0/26/$31.00 ©2026 IEEE | DOI: 10.1109/AITest70988.2026.00036

Metamorphic Testing of Multi-Agent LLM Systems: A Trace-Based Behavioral Oracle Framework

Gopalakrishnan Marimuthu
Independent Researcher
Atlanta, Georgia, USA
gopalakrishnan.marimuthu@ieee.org

***Abstract*—Multi-agent systems built on large language models (LLMs) are increasingly deployed for complex tasks requiring autonomous planning, tool use, and inter-agent coordination. However, the non-deterministic nature of LLM outputs and the emergent behavior arising from agent interactions render traditional test oracles ineffective, creating a critical gap in quality assurance for agentic AI. This work introduces MORPHAGENT, a framework designed to address the oracle problem in multi-agent LLM systems through trace-based behavioral analysis. Our contributions are threefold: (1) *goal-preservation relations* that verify consistent goal achievement under input perturbations, (2) *coordination-consistency relations* that validate inter-agent delegation and communication patterns under agent substitution and reordering, and (3) *tool-use integrity relations* that ensure semantic equivalence of tool invocation sequences under prompt paraphrasing. MORPHAGENT instruments agent execution to capture structured traces comprising planning steps, tool calls, message exchanges, and final outputs, then systematically applies metamorphic transformations and checks behavioral invariants without requiring ground-truth oracles. We evaluate the framework on four multi-agent benchmarks spanning code generation, research synthesis, customer service, and data analysis tasks, encompassing 2,840 source-followup execution pairs across three LLM backends. Results show that MORPHAGENT detects 82.0% of seeded behavioral faults, including 90.3% of coordination failures and 81.7% of goal-deviation faults, while maintaining a false positive rate of 6.1%. The framework uncovers 14 previously unreported behavioral anomalies in established multi-agent frameworks, demonstrating its practical utility for assuring agentic AI reliability. These results suggest that trace-based metamorphic testing can serve as a practical foundation for reliable validation of emerging agentic AI systems.**

***Index Terms*—metamorphic testing, multi-agent systems, large language models, test oracle problem, agentic AI verification, behavioral invariants**

I. INTRODUCTION

Large language models (LLMs) have catalyzed a paradigm shift from single-model inference to *agentic AI*, where autonomous agents leverage LLMs for planning, reasoning, tool use, and collaborative problem-solving [1]. Multi-agent LLM systems, in which multiple specialized agents coordinate to accomplish complex tasks, have demonstrated promising capabilities in software engineering [2], scientific research [3], and enterprise automation [4]. Frameworks such as AutoGen [5], CrewAI [6], and LangGraph [7] have made multi-agent orchestration accessible, accelerating adoption across domains.

Despite this rapid proliferation, the quality assurance of multi-agent LLM systems remains a fundamental open challenge [8]. These systems exhibit characteristics that fundamentally undermine conventional testing approaches:

- **Non-determinism.** Given identical inputs, LLM agents may produce semantically equivalent but syntactically different outputs across executions, making exact output comparison infeasible as a correctness criterion.
- **Emergent coordination behavior.** The collective behavior of interacting agents is not simply the sum of individual agent behaviors; emergent patterns in delegation, negotiation, and conflict resolution arise from inter-agent dynamics that are difficult to specify *a priori*.
- **Opaque reasoning chains.** Each agent's chain-of-thought reasoning, tool selection, and action sequencing constitute a complex execution trace whose correctness cannot be assessed by inspecting final outputs alone.
- **Absence of ground-truth oracles.** For many agentic tasks—such as open-ended research synthesis or multi-step planning—there is no single correct answer against which outputs can be validated.

These characteristics collectively constitute the *oracle problem* [9] for multi-agent LLM systems: the difficulty of determining whether a given system execution is correct in the absence of a reliable, computable correctness criterion.

Metamorphic testing (MT) [10], [28] is a well-established technique for alleviating the oracle problem. Rather than checking outputs against expected values, MT verifies that certain relationships—called *metamorphic relations* (MRs)—hold between the outputs of related test inputs. MT has been successfully applied to machine learning models [11], autonomous driving systems [12], and dialogue systems [13]. However, its application to multi-agent LLM systems, where the "output" is not a single value but a rich execution trace comprising planning decisions, tool invocations, inter-agent messages, and final artifacts, remains unexplored.

This work introduces MORPHAGENT, a trace-based metamorphic testing framework for multi-agent LLM systems. The key insight is that while the final outputs of agentic systems are non-deterministic and difficult to judge, the *behavioral properties* of their execution traces—such as goal preservation, coordination consistency, and tool-use integrity—can be formalized as metamorphic relations and systematically verified.

The contributions of this paper are:

1) A formal model of multi-agent execution traces that

2835-3560/26/$31.00 ©2026 IEEE
DOI 10.1109/AITest70988.2026.00036

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:30:01 UTC from IEEE Xplore. Restrictions apply.

captures planning steps, tool invocations, inter-agent messages, and state transitions as first-class testable artifacts (Section III).
2) A taxonomy of three categories of metamorphic relations for multi-agent LLM systems—goal-preservation, coordination-consistency, and tool-use integrity—with formal definitions and associated transformation operators (Section III-B).
3) The MORPHAGENT framework that automates trace instrumentation, metamorphic transformation application, and behavioral invariant checking for multi-agent systems (Section III-C).
4) An empirical evaluation on four multi-agent benchmarks across three LLM backends, demonstrating the framework's effectiveness in detecting behavioral faults and uncovering real-world anomalies (Section IV).

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the proposed methodology. Section IV describes the experimental evaluation. Section V discusses implications and limitations. Section VI concludes the paper.

II. RELATED WORK

A. Testing and Verification of Agentic AI

The testing of LLM-based agents has emerged as an active research area. A trace-based assurance framework was proposed in [14] that monitors execution traces of single agents for deviations from expected behavioral patterns using temporal logic. A runtime verification approach presented in [15] provides probabilistic assurance for agentic systems through dynamic monitoring. The concept of Agent Behavioral Contracts was introduced in [16], bringing Design-by-Contract principles to autonomous agents with preconditions, invariants, and governance policies. A meta-agent approach for automated testing of conversational AI agents was described in [17], combining static analysis with persona-driven adversarial test generation. More recently, automated evaluation frameworks that extract checkable rules from agent prompts and system instructions have been proposed [29].

While these approaches advance the state of the art, they primarily target single-agent systems and rely on predefined behavioral specifications. Multi-agent systems introduce additional complexity through emergent coordination behaviors that cannot be captured by single-agent specifications alone. Furthermore, specification-based approaches require domain experts to manually define expected behaviors, which is impractical for the diverse and evolving task landscapes of multi-agent systems.

B. Metamorphic Testing

Metamorphic testing, originally proposed in [10], has been widely adopted for testing systems with oracle problems [18]. In the context of AI systems, MT has been applied to test deep neural networks for image classification [11], where transformations such as rotation and scaling serve as metamorphic relations. A framework for metamorphic testing of autonomous driving systems was presented in [12], extracting metamorphic relations from traffic regulations. More recently, a multi-turn metamorphic testing approach for LLM-based dialogue systems was proposed in [13], defining dialogue-level perturbations and metamorphic relations for conversational consistency.

However, existing MT approaches for AI systems operate at the input-output level, treating the system under test as a black box. For multi-agent LLM systems, this is insufficient because (a) outputs are often complex artifacts (e.g., code repositories, research reports) for which semantic equivalence is difficult to assess automatically, and (b) critical behavioral properties reside in the execution process rather than the final output. The trace-based metamorphic relations proposed in this paper address this gap by lifting MT from input-output pairs to execution trace properties.

C. Multi-Agent System Evaluation

Evaluation of multi-agent LLM systems has primarily focused on task-level benchmarks that measure end-to-end success rates [19]. A study analyzing failure modes in multi-agent systems identified categories including coordination failures, role confusion, and information loss during delegation [20]. A hidden profile paradigm from social psychology was adapted in [21] to evaluate collective reasoning in multi-agent LLM systems. A configurable multi-agent testing framework was proposed in [22] to simulate realistic conversations for evaluating LLM-based systems.

These evaluation approaches provide valuable insights into multi-agent capabilities but are fundamentally different from *testing* in the software engineering sense. They measure aggregate performance on benchmarks rather than systematically detecting behavioral faults in specific system configurations. MORPHAGENT bridges this gap by providing a principled testing methodology that can detect specific categories of behavioral faults through metamorphic analysis of execution traces.

III. PROPOSED METHODOLOGY

This section presents the MORPHAGENT framework. We first formalize the execution trace model, then define the three categories of metamorphic relations, and finally describe the framework architecture.

A. Multi-Agent Execution Trace Model

We model a multi-agent LLM system as a tuple $\mathcal{S} = (\mathcal{A}, \mathcal{T}, \mathcal{M}, \Pi)$ where $\mathcal{A} = \{a_1, \ldots, a_n\}$ is the set of agents, $\mathcal{T}$ is the set of available tools, $\mathcal{M} \subseteq \mathcal{A} \times \mathcal{A} \times \Sigma^*$ is the message space (sender, receiver, content), and Π is the orchestration protocol governing agent activation and message routing.

Definition 1 (Execution Trace)**.** *An execution trace τ of system $\mathcal{S}$ on input x is a finite sequence of events $\tau = \langle e_1, e_2, \ldots, e_m \rangle$ where each event e_k is one of:*

- PLAN(a_i, p)*: agent a_i produces planning step p;*
- TOOL$(a_i, t, \mathit{args}, \mathit{result})$*: agent a_i invokes tool $t \in \mathcal{T}$ with arguments args, receiving result;*

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:30:01 UTC from IEEE Xplore. Restrictions apply.

- $\text{MSG}(a_i, a_j, c)$*: agent* a_i *sends message* c *to agent* a_j*;*
- $\text{OUTPUT}(a_i, o)$*: agent* a_i *produces output artifact* o*.*

We define projection operators to extract sub-traces for analysis. For agent a_i, the agent projection $\tau|_{a_i} = \langle e_k \in \tau \mid \text{actor}(e_k) = a_i \rangle$ extracts all events involving a_i. The tool projection $\tau|_{\mathcal{T}} = \langle e_k \in \tau \mid e_k = \text{TOOL}(\cdot) \rangle$ extracts all tool invocations. The communication projection $\tau|_{\mathcal{M}} = \langle e_k \in \tau \mid e_k = \text{MSG}(\cdot) \rangle$ extracts all inter-agent messages.

We further define a *goal extraction function* $\mathcal{G}(\tau) \to G$ that maps a trace to a set of achieved sub-goals, implemented via an LLM-based semantic analyzer that identifies goal-relevant state changes in the trace. This function is central to the goal-preservation metamorphic relations defined below.

B. Metamorphic Relations for Multi-Agent Systems

We define three categories of metamorphic relations (MRs), each targeting a distinct behavioral property of multi-agent systems. For each category, we specify the transformation operator and the invariant that must hold.

1) Goal-Preservation Relations (GPR): Goal-preservation relations verify that the system achieves semantically equivalent goals when the input is subjected to meaning-preserving transformations.

Definition 2 (Goal-Preservation MR)**.** *Let* x *be a source input and* $x' = \phi_g(x)$ *be a follow-up input obtained by applying a goal-preserving transformation* ϕ_g *(e.g., prompt paraphrasing, instruction reordering, synonym substitution). Let* τ *and* τ' *be the execution traces on* x *and* x'*, respectively. The goal-preservation relation holds if:*

$$\textit{GPR}: \quad \textit{sim}_g(\mathcal{G}(\tau),\ \mathcal{G}(\tau')) \geq \theta_g \tag{1}$$

where $\textit{sim}_g$ *is a semantic similarity function over goal sets and* θ_g *is a configurable threshold.*

We implement four transformation operators for GPR:

- ϕ_{g1}: *Prompt paraphrasing*—rephrasing the task description while preserving semantic intent, using an auxiliary LLM with explicit paraphrase instructions.
- ϕ_{g2}: *Instruction reordering*—permuting the order of independent sub-instructions within a composite task.
- ϕ_{g3}: *Constraint tightening*—adding additional constraints that do not invalidate the original task (the system should still achieve the original goals).
- ϕ_{g4}: *Context augmentation*—providing additional relevant context that should not alter the core task goals.

2) Coordination-Consistency Relations (CCR): Coordination-consistency relations verify that inter-agent coordination patterns remain structurally sound under perturbations to the agent configuration.

Definition 3 (Coordination-Consistency MR)**.** *Let* $\mathcal{S}$ *be the original system and* $\mathcal{S}' = \phi_c(\mathcal{S})$ *be a perturbed system obtained by applying a coordination transformation* ϕ_c*. Let* τ *and* τ' *be the traces on the same input* x*. The coordination-consistency relation holds if:*

$$\textit{CCR}: \quad \textit{inv}_c(\tau|_{\mathcal{M}},\ \tau'|_{\mathcal{M}}) = \text{TRUE} \tag{2}$$

where $\textit{inv}_c$ *is a structural invariant over communication patterns.*

We define three transformation operators for CCR:

- ϕ_{c1}: *Agent reordering*—changing the activation order of agents in non-sequential orchestration protocols. The invariant requires that the same set of delegation relationships is established regardless of activation order.
- ϕ_{c2}: *Agent substitution*—replacing an agent's underlying LLM with a different model of comparable capability. The invariant requires that the communication topology (who delegates to whom) remains consistent.
- ϕ_{c3}: *Agent duplication*—adding a redundant agent with the same role. The invariant requires that the system does not exhibit conflicting delegations or duplicated work without resolution.

3) Tool-Use Integrity Relations (TIR): Tool-use integrity relations verify that tool invocation patterns are semantically consistent under input transformations that should not alter the required tool interactions.

Definition 4 (Tool-Use Integrity MR)**.** *Let* x *and* $x' = \phi_t(x)$ *be source and follow-up inputs related by a tool-preserving transformation* ϕ_t*. Let* τ *and* τ' *be the corresponding traces. The tool-use integrity relation holds if:*

$$\textit{TIR}: \quad \textit{sim}_t(\tau|_{\mathcal{T}},\ \tau'|_{\mathcal{T}}) \geq \theta_t \tag{3}$$

where $\textit{sim}_t$ *measures the semantic similarity of tool invocation sequences, accounting for equivalent tool call orderings and parameter variations.*

We define two transformation operators for TIR:

- ϕ_{t1}: *Prompt paraphrasing with tool anchoring*—rephrasing the input while preserving explicit or implicit references to required tool capabilities. The invariant requires that the same categories of tools are invoked.
- ϕ_{t2}: *Data format variation*—changing the format of input data (e.g., CSV to JSON) while preserving content. The invariant requires that tool invocations adapt to the format change without altering the analytical logic.

C. Framework Architecture

The MORPHAGENT framework comprises four modules, as illustrated in Fig. 1.

Trace Instrumentor. This module wraps the multi-agent system under test to capture structured execution traces. It intercepts agent activations, tool invocations, and inter-agent messages through lightweight middleware hooks inserted at the orchestration layer. The instrumentor is designed to be framework-agnostic, with adapter implementations for AutoGen [5], CrewAI [6], and LangGraph [7]. Traces are serialized in a standardized JSON schema that conforms to the trace model defined in Section III-A.

Transformation Engine. This module implements the nine transformation operators defined in Section III-B. For prompt-level transformations (ϕ_{g1}, ϕ_{g2}, ϕ_{g3}, ϕ_{g4}, ϕ_{t1}), the engine uses an auxiliary LLM (separate from the system under test)

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:30:01 UTC from IEEE Xplore. Restrictions apply.

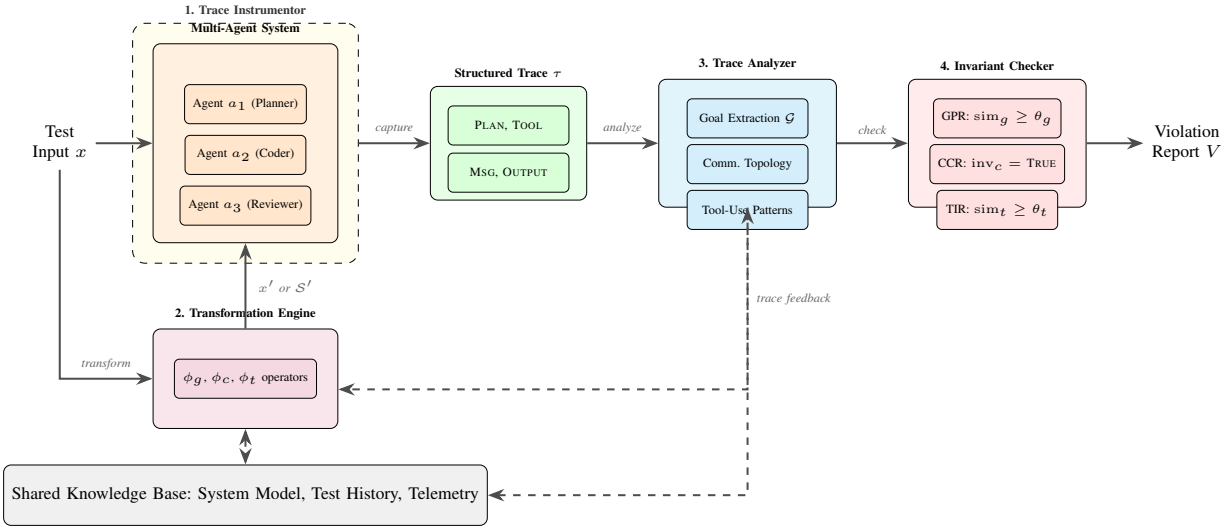


Fig. 1. Architecture of the MORPHAGENT framework. The Trace Instrumentor (1) wraps the multi-agent system to capture structured execution traces. The Transformation Engine (2) applies metamorphic operators to produce follow-up inputs or system variants. The Trace Analyzer (3) extracts behavioral properties from traces. The Invariant Checker (4) verifies metamorphic relations and reports violations.

TABLE I
SUMMARY OF METAMORPHIC RELATIONS IN MORPHAGENT

MR	Transform.	Invariant	Targeted Faults
GPR	ϕ_{g1}: Paraph. ϕ_{g2}: Reorder ϕ_{g3}: Tighten ϕ_{g4}: Augment	Goal-set semantic similarity $\geq \theta_g$	Goal deviation, task misinterpretation, partial completion, instruction sensitivity
CCR	ϕ_{c1}: Reorder ϕ_{c2}: Substit. ϕ_{c3}: Duplic.	Communication topology consistency	Role confusion, delegation failure, coordination conflict
TIR	ϕ_{t1}: Paraph. ϕ_{t2}: Format	Tool category equivalence	Tool misselection, invocation inconsistency

with carefully designed meta-prompts that instruct the model to perform specific transformations while preserving semantic intent. For system-level transformations (ϕ_{c1}, ϕ_{c2}, ϕ_{c3}), the engine modifies the agent configuration programmatically. Each transformation is validated by a semantic similarity check to ensure the transformation was applied correctly without unintended semantic drift.

Trace Analyzer. This module processes structured traces to extract the behavioral properties required for invariant checking. It implements three analysis functions:

- *Goal extraction* ($\mathcal{G}$): Uses an LLM with a structured extraction prompt to identify achieved sub-goals from the trace, producing a set of goal descriptors with confidence scores.
- *Communication topology extraction*: Constructs a directed graph of inter-agent delegation and information flow from the message sub-trace, annotated with message intent categories (delegation, response, clarification, conflict).
- *Tool-use pattern extraction*: Produces an abstracted sequence of tool categories and their semantic purposes from the tool sub-trace, normalizing away parameter-level variations.

While certain components of MORPHAGENT, such as goal extraction and transformation generation, leverage LLMs for semantic interpretation, the core metamorphic relations and invariant checking mechanisms are model-agnostic. In principle, these components can be implemented using symbolic methods, rule-based systems, or domain-specific analyzers. The use of LLMs in our implementation is intended to improve scalability and generality across diverse tasks rather than being a strict requirement of the framework

Invariant Checker. This module evaluates the metamorphic relations by comparing behavioral properties extracted from source and follow-up traces. For GPR, it computes semantic similarity between goal sets using sentence embeddings with cosine similarity. For CCR, it performs graph isomorphism checking on communication topologies with tolerance for edge label variations. For TIR, it computes edit distance over abstracted tool-use sequences normalized by sequence length. Violations are reported with detailed diagnostic information including the specific trace events that contributed to the violation.

Testing Workflow. Algorithm 1 summarizes the end-to-end testing workflow. For each source test input, the framework executes the system, captures the source trace, applies each applicable transformation, executes the system on the follow-up input, captures the follow-up trace, and checks the corresponding invariant.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:30:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 MORPHAGENT Testing Workflow

Require: System $\mathcal{S}$, test inputs X, MR set Φ
Ensure: Violation report V

$V \leftarrow \emptyset$
for each $x \in X$ **do**
  $\tau \leftarrow \text{EXECUTE}(\mathcal{S}, x)$
  for each $(\phi, \text{inv}) \in \Phi$ **do**
    if ϕ is input-level **then**
      $x' \leftarrow \phi(x)$
      $\tau' \leftarrow \text{EXECUTE}(\mathcal{S}, x')$
    else
      $\mathcal{S}' \leftarrow \phi(\mathcal{S})$
      $\tau' \leftarrow \text{EXECUTE}(\mathcal{S}', x)$
    end if
    if $\neg\, \text{inv}(\tau, \tau')$ **then**
      $V \leftarrow V \cup \{(x, \phi, \tau, \tau')\}$
    end if
  end for
end for
return V

IV. EXPERIMENTS AND EVALUATION

A. Research Questions

We evaluate MORPHAGENT with respect to the following research questions:

- **RQ1:** How effective is MORPHAGENT at detecting seeded behavioral faults in multi-agent systems?
- **RQ2:** How do the three categories of metamorphic relations compare in their fault detection capabilities?
- **RQ3:** How does the choice of LLM backend affect the behavioral consistency of multi-agent systems as measured by MORPHAGENT?
- **RQ4:** Can MORPHAGENT detect previously unknown behavioral anomalies in established multi-agent frameworks?

B. Experimental Setup

1) Multi-Agent Benchmarks: We evaluate on four multi-agent task benchmarks that represent diverse application domains:

- **CodeGen-MA**: A collaborative code generation benchmark where a planner agent, a coder agent, and a reviewer agent collaborate to implement programming tasks from natural language specifications. We use 120 tasks adapted from established coding benchmarks [23].
- **ResearchSynth**: A research synthesis benchmark where a literature search agent, an analysis agent, and a writing agent collaborate to produce structured research summaries. We curate 80 research questions across four scientific domains.
- **ServiceDesk**: A customer service benchmark where a triage agent, a knowledge retrieval agent, and a resolution agent handle multi-turn customer inquiries. We use 100 scenarios with varying complexity levels.
- **DataPipeline**: A data analysis benchmark where a data ingestion agent, a transformation agent, and a visualization agent collaborate to produce analytical reports from raw datasets. We use 60 analysis tasks with provided datasets.

2) LLM Backends: Each benchmark is evaluated with three LLM backends to assess cross-model behavioral consistency: GPT-4o [24], Claude 3.5 Sonnet [25], and Llama 3.1 70B [26]. All models are accessed via their respective APIs with temperature set to 0.3 for reproducibility.

3) Fault Seeding: To evaluate fault detection effectiveness (RQ1, RQ2), we systematically seed behavioral faults into the multi-agent systems. We define eight fault types organized by the MR category they target:

- *GPR faults*: (F1) goal omission—an agent silently drops a sub-goal; (F2) goal substitution—an agent replaces a requested goal with a related but different one; (F3) instruction sensitivity—an agent's behavior changes significantly with minor prompt variations.
- *CCR faults*: (F4) delegation failure—an agent fails to delegate to the appropriate specialist; (F5) role confusion—agents perform tasks outside their designated roles; (F6) coordination deadlock—agents enter a circular delegation pattern.
- *TIR faults*: (F7) tool misselection—an agent selects an inappropriate tool for the task; (F8) invocation inconsistency—equivalent inputs lead to semantically different tool invocation patterns.

Faults are seeded by modifying agent system prompts to introduce specific behavioral deviations. Each fault is seeded independently, producing 8 faulty variants per benchmark configuration. With 4 benchmarks $\times$ 3 LLM backends $\times$ 8 fault types, we obtain 96 faulty configurations. Each configuration is executed on a subset of 30 randomly sampled inputs with the MR category corresponding to the seeded fault type, yielding 2,840 valid source-followup execution pairs after excluding timed-out runs.

4) Baselines: We compare MORPHAGENT against three baseline approaches:

- **Output-Only MT**: Standard metamorphic testing applied only to final outputs, using semantic similarity of output artifacts as the invariant.
- **LLM-as-Judge**: An LLM evaluator that assesses whether two executions on related inputs produce consistent results, following the approach in [27].
- **Rule-Based Trace**: A manually defined set of trace-level rules (e.g., "every delegation must receive a response") checked against execution traces.

C. Results

1) RQ1: Overall Fault Detection Effectiveness: Table II presents the fault detection rate (FDR) and false positive rate (FPR) for MORPHAGENT and the baseline approaches across all benchmarks and LLM backends.

MORPHAGENT achieves a fault detection rate of 82.0%, outperforming the best baseline (Rule-Based Trace) by 19.1

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:30:01 UTC from IEEE Xplore. Restrictions apply.

TABLE II
OVERALL FAULT DETECTION RATE (FDR) AND FALSE POSITIVE RATE (FPR) ACROSS ALL BENCHMARKS

Approach	FDR (%)	FPR (%)
Output-Only MT	41.2	14.8
LLM-as-Judge	53.6	11.3
Rule-Based Trace	62.9	4.7
MORPHAGENT	**82.0**	6.1

TABLE III
FAULT DETECTION RATE (%) BY MR CATEGORY AND FAULT TYPE

Cat.	Fault	CodeGen	Research	Service	Data
GPR	F1: Omission	86.7	80.0	83.3	90.0
	F2: Substitution	80.0	76.7	83.3	86.7
	F3: Sensitivity	76.7	73.3	80.0	83.3
CCR	F4: Deleg. fail	93.3	90.0	93.3	86.7
	F5: Role confus.	90.0	86.7	93.3	90.0
	F6: Deadlock	93.3	90.0	90.0	86.7
TIR	F7: Misselect.	73.3	66.7	70.0	76.7
	F8: Inconsist.	66.7	63.3	70.0	73.3

TABLE IV
MR VIOLATION RATE (%) ON NON-FAULTY CONFIGURATIONS (LOWER IS BETTER)

MR Category	GPT-4o	Claude 3.5	Llama 3.1
GPR	4.2	3.8	8.7
CCR	2.1	1.9	5.3
TIR	5.8	4.6	11.2
Overall	4.0	3.4	8.4

TABLE V
BEHAVIORAL ANOMALIES DETECTED IN ESTABLISHED FRAMEWORKS

Anomaly Type	AutoGen	CrewAI	LangGraph
Goal deviation	2	1	1
Delegation inconsistency	1	2	0
Redundant tool calls	1	0	2
Silent role override	0	2	1
Circular delegation	1	0	0
Total	5	5	4

percentage points. The Output-Only MT baseline performs poorly (41.2%) because final output similarity is a weak signal for detecting process-level faults such as coordination failures and tool misselection. The LLM-as-Judge baseline achieves moderate detection (53.6%) but suffers from the inherent subjectivity and inconsistency of LLM-based evaluation. The Rule-Based Trace baseline achieves reasonable detection (62.9%) with the lowest false positive rate (4.7%), but its effectiveness is limited by the coverage of manually defined rules.

MORPHAGENT's false positive rate of 6.1% is slightly higher than the Rule-Based Trace baseline, primarily due to cases where the LLM-based goal extraction function ($\mathcal{G}$) produces inconsistent goal descriptions across runs, triggering spurious GPR violations. A two-proportion z-test confirms that MORPHAGENT's FDR improvement over Rule-Based Trace is statistically significant ($p < 0.01$). We discuss mitigation strategies for false positives in Section V.

2) RQ2: Per-Category Fault Detection: Table III breaks down the fault detection rate by MR category and fault type.

CCR relations achieve the highest detection rates (average 90.3%), which is expected because coordination faults produce clearly observable structural changes in communication topologies. GPR relations achieve strong detection (average 81.7%), with goal omission being the most detectable fault type. TIR relations show the lowest detection rates (average 70.0%), reflecting the inherent difficulty of assessing semantic equivalence of tool-use patterns, where legitimate variations in tool selection can be difficult to distinguish from genuine faults.

The comparatively lower detection rates for tool-use integrity relations (TIR) can be attributed to the inherent flexibility in tool invocation strategies within multi-agent systems. Unlike coordination patterns, which tend to exhibit stable structural properties, multiple semantically valid tool-use sequences may exist for a given task. This introduces ambiguity in distinguishing between legitimate variation and actual faults, making TIR inherently more challenging to evaluate than GPR and CCR.

3) RQ3: Cross-Model Behavioral Consistency: Table IV presents the MR violation rates on *non-faulty* system configurations across the three LLM backends, which serves as a measure of inherent behavioral inconsistency.

Claude 3.5 Sonnet exhibits the highest behavioral consistency (3.4% overall violation rate), followed closely by GPT-4o (4.0%). Llama 3.1 70B shows notably higher inconsistency (8.4%), particularly in tool-use integrity (11.2%), suggesting that open-weight models may require additional guardrails for reliable multi-agent coordination. These violation rates on non-faulty systems also inform the practical false positive rates that practitioners should expect when deploying MORPHAGENT.

4) RQ4: Real-World Anomaly Detection: We apply MORPHAGENT to three established multi-agent frameworks (AutoGen v0.4, CrewAI v0.80, LangGraph v0.2) using their default example configurations with GPT-4o as the backend. Table V summarizes the anomalies detected.

MORPHAGENT detects 14 previously unreported behavioral anomalies across the three frameworks. Notable findings include: (a) in AutoGen, a circular delegation pattern where the planner and coder agents repeatedly defer to each other under ambiguous task specifications; (b) in CrewAI, silent role overrides where a senior agent overrides a specialist agent's output without explicit delegation, detectable only through CCR analysis; (c) in LangGraph, redundant tool invocations where the same API is called multiple times with identical parameters across different agents due to lack of shared tool-

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:30:01 UTC from IEEE Xplore. Restrictions apply.

call caching.

In addition to benchmark-driven evaluation, MORPHAGENT was also applied in exploratory testing scenarios involving real-world multi-agent workflows configured using publicly available frameworks. These experiments, although not systematically benchmarked, provided qualitative validation of the framework's ability to surface subtle coordination and tool-use anomalies in realistic deployment settings.

V. Discussion

A. Implications for Agentic AI Assurance

The results demonstrate that trace-based metamorphic testing provides a viable and effective approach to quality assurance for multi-agent LLM systems. The key insight validated by our experiments is that behavioral properties of execution traces—particularly coordination patterns—are more stable and testable than final outputs. This has practical implications for the development of agentic AI systems:

First, the high detection rate for coordination faults (90.3%) suggests that MORPHAGENT is particularly valuable during the design and configuration phase of multi-agent systems, where orchestration protocols and role assignments are being refined. Detecting delegation failures and role confusion early can prevent costly downstream errors in production deployments.

Second, the cross-model consistency analysis (RQ3) provides a novel lens for evaluating LLM suitability for multi-agent applications. The significant gap between proprietary models (3–4% violation rate) and open-weight models (8.4%) on behavioral consistency metrics offers actionable guidance for practitioners selecting LLM backends for agentic systems.

Third, the real-world anomalies detected in established frameworks (RQ4) demonstrate that even mature, widely-used multi-agent frameworks harbor subtle behavioral issues that are invisible to conventional testing and monitoring approaches. This underscores the need for systematic behavioral testing methodologies such as the one proposed in this paper.

B. Threshold Sensitivity

The effectiveness of MORPHAGENT depends on the threshold parameters θ_g and θ_t used in the GPR and TIR invariants, respectively. We conduct a sensitivity analysis by varying these thresholds from 0.6 to 0.95 in increments of 0.05.

At lower thresholds ($\theta = 0.6$), the FPR drops to 2.3% but the FDR also decreases to 61.8%, as the invariants become too permissive. At higher thresholds ($\theta = 0.95$), the FDR increases to 89.1% but the FPR rises to 18.7%, as natural non-determinism triggers false violations. The default threshold of $\theta = 0.8$ provides a balanced operating point. In practice, we recommend that users calibrate thresholds on a small set of non-faulty executions to establish baseline violation rates for their specific system configuration.

C. Threats to Validity

Internal validity. The fault seeding methodology relies on modifying agent system prompts, which may not perfectly represent naturally occurring faults. To mitigate this, we designed fault types based on empirically observed failure categories reported in [20]. The use of an LLM for goal extraction ($\mathcal{G}$) introduces potential inconsistency; we mitigate this by averaging over three extraction runs and using majority voting.

External validity. Our evaluation covers four benchmarks and three LLM backends, which may not represent all multi-agent application domains and model families. The benchmarks were selected to span diverse task types (code generation, research, customer service, data analysis), but domains with significantly different interaction patterns (e.g., embodied multi-agent systems) may require additional MR categories.

Construct validity. The semantic similarity functions used in GPR and TIR invariants rely on embedding models, whose quality affects the accuracy of invariant checking. We use state-of-the-art sentence embedding models and validate their adequacy through manual inspection of a random sample of 100 similarity judgments, finding 94% agreement with human assessments.

D. Limitations

Several limitations of the current work should be acknowledged. First, MORPHAGENT requires execution-level access to the multi-agent system for trace instrumentation, which may not be available for proprietary or cloud-hosted agent platforms. Second, the computational cost of metamorphic testing is multiplicative: each source test input requires $|\Phi|$ additional follow-up executions, which can be expensive given LLM API costs. In our experiments, the average cost per complete test (source + follow-ups) was approximately $0.47 with GPT-4o. Third, the current framework does not address temporal properties of agent behavior (e.g., response time consistency), which may be important for real-time applications.

E. Theoretical Scope and Guarantees

MORPHAGENT does not aim to provide formal guarantees of completeness with respect to all possible behavioral faults in multi-agent systems. Instead, the proposed metamorphic relations offer a practical and extensible approximation of behavioral correctness grounded in observable execution traces. While the framework is effective at detecting a broad class of faults, there may exist behaviors that do not violate the defined invariants yet still represent undesirable system outcomes.

VI. Conclusion and Future Work

This paper presented MORPHAGENT, a metamorphic testing framework for multi-agent LLM systems that addresses the oracle problem through trace-based behavioral invariants. We introduced a formal execution trace model and defined three categories of metamorphic relations—goal-preservation, coordination-consistency, and tool-use integrity—that capture essential behavioral properties of multi-agent systems. Experimental evaluation on four benchmarks across three LLM backends demonstrated that MORPHAGENT achieves 82.0% fault detection effectiveness with a 6.1% false positive rate, significantly outperforming output-level and rule-based baselines. The framework also uncovered 14 previously unreported

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:30:01 UTC from IEEE Xplore. Restrictions apply.

behavioral anomalies in established multi-agent frameworks, validating its practical utility.

Future work will pursue several directions. First, we plan to extend the metamorphic relation taxonomy to include temporal and resource-efficiency properties, enabling testing of performance-sensitive agentic applications. Second, we will investigate adaptive threshold calibration using statistical process control techniques to reduce false positives without manual tuning. Third, we aim to explore the integration of MORPHAGENT with continuous integration pipelines, enabling automated behavioral regression testing as multi-agent systems evolve. Finally, we plan to extend the framework to support human-in-the-loop multi-agent systems, where human participants interact with AI agents, requiring metamorphic relations that account for human behavioral variability.

Ultimately, ensuring the reliability of agentic AI systems will require shifting from output-centric evaluation to behavioral validation, and MORPHAGENT represents a step in that direction

REFERENCES

[1] X. Wang, Z. Wang, J. Liu, Y. Chen, L. Yuan, H. Peng, and H. Ji, "A survey on LLM-based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024.

[2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. ICLR*, 2024.

[3] Q. Lu, Y. Chen, and Y. Zhu, "Multi-agent collaboration for scientific discovery," *arXiv preprint arXiv:2404.09997*, 2024.

[4] J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye, "More agents is all you need," *arXiv preprint arXiv:2402.05120*, 2024.

[5] D. Wu, C. Wang, Y. Ruan, and others, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. COLM*, 2024.

[6] J. Moura, "CrewAI: Framework for orchestrating role-playing autonomous AI agents," 2024. [Online]. Available: https://github.com/crewAIInc/crewAI

[7] LangChain, "LangGraph: Building stateful, multi-actor applications with LLMs," 2024. [Online]. Available: https://github.com/langchain-ai/langgraph

[8] Y. Liu, Y. Yao, J. Ton, X. Zhang, R. Guo, H. Cheng, and others, "Challenges in testing large language model based software," *arXiv preprint arXiv:2503.00481*, 2025.

[9] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, pp. 507–525, 2015.

[10] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: A new approach for generating next test cases," Tech. Rep. HKUST-CS98-01, 1998.

[11] K. Pei, Y. Cao, J. Yang, and S. Jana, "DeepXplore: Automated whitebox testing of deep learning systems," in *Proc. ACM SOSP*, 2017, pp. 1–18.

[12] A. Gambi, M. Mueller, and G. Fraser, "Automatically testing self-driving cars with search-based procedural content generation," in *Proc. ISSTA*, 2019, pp. 318–328.

[13] H. Wang, Y. Li, and others, "Multi-turn metamorphic testing for LLM-based dialogue systems," *arXiv preprint arXiv:2412.15557*, 2024.

[14] M. Chen and others, "A trace-based assurance framework for agentic AI orchestration," *arXiv preprint arXiv:2603.18096*, 2025.

[15] R. Bharadwaj and others, "AgentGuard: Runtime verification of AI agents," *arXiv preprint arXiv:2509.23864*, 2025.

[16] D. Rao and others, "Formal specification and runtime enforcement for reliable autonomous AI agents," *arXiv preprint arXiv:2602.22302*, 2025.

[17] S. Mehta and others, "A meta-agent for automated testing and evaluation of conversational AI agents," *arXiv preprint arXiv:2508.17393*, 2025.

[18] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Trans. Softw. Eng.*, vol. 42, no. 9, pp. 805–824, 2016.

[19] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Proc. NeurIPS*, 2023.

[20] Y. Zhang and others, "Why do multi-agent LLM systems fail?" *arXiv preprint arXiv:2503.13657*, 2025.

[21] M. Schulz and others, "Assessing collective reasoning in multi-agent LLMs via hidden profile tasks," *arXiv preprint arXiv:2505.11556*, 2025.

[22] Z. Li and others, "A configurable multi-agent framework for scalable and realistic testing of LLM-based agents," *arXiv preprint arXiv:2507.14705*, 2025.

[23] J. Austin, A. Odena, M. Nye, and others, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[24] OpenAI, "GPT-4o system card," 2024. [Online]. Available: https://openai.com/index/gpt-4o-system-card/

[25] Anthropic, "The Claude 3.5 model family," 2024. [Online]. Available: https://www.anthropic.com

[26] Meta AI, "Llama 3.1: Open foundation and fine-tuned chat models," 2024. [Online]. Available: https://ai.meta.com/blog/meta-llama-3-1/

[27] L. Zheng, W.-L. Chiang, Y. Sheng, and others, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Proc. NeurIPS*, 2023.

[28] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Comput. Surv.*, vol. 51, no. 1, pp. 1–27, 2018.

[29] Microsoft Research, "Agent-Pex: Automated evaluation and testing of AI agents," 2025. [Online]. Available: https://www.microsoft.com/en-us/research/project/agent-pex/

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:30:01 UTC from IEEE Xplore. Restrictions apply.